

Lab5 Tree2Quad

1. tree2quad

转换的入口，初始化 visitor 状态，创建空 QuadProgram，调用 prog->accept(v) 触发递归访问。把树结构的 Program 转换为顺序四元组。

2. Visit

2.1 Program

遍历 funcdecllist，逐个翻译函数。顺便统计 last_label 和 last_temp。

2.2 FuncDecl

基于函数在IR阶段的 last_temp/last_label，设置继续分配的 next_temp/next_label。新建一个 vector<QuadStm*> 作为当前函数所有 Quad 语句的返回值。生成函数入口作为第一个 Quad 语句。然后递归翻译函数内的 Stm。得到的 visit_result 和入口 entry 打包成 QuadBlock，再封装为 QuadFuncDecl 添加到 Program 的 List 中。然后更新函数的 last_temp/last_label。

2.3 Jump

根据 Label 初始化 QuadJump。

2.4 Cjump

先分别翻译 left 和 right，将 output_term 分别记为 left_term 和 right_term。然后添加到 use 的 set 中。打包为 QuadCJump 返回。

2.5 Move

把一个树形赋值，按 `dst<-src` 形态映射成最合适的四元组类型：

`Temp <- Call` → MOVE_CALL

`Temp <- ExtCall` → MOVE_EXTCALL

`Temp <- Mem` → LOAD

`Temp <- Binop` → MOVE_BINOP / PTR_CALC

`Mem <- x` → STORE

其他 `Temp <- x` → MOVE

首先判断，如果左值是 mem，转换为 mem 再visit，翻译得到 dst_term。

否则直接 visit `move->dst`。

2.5.1 Temp<-Call

把 src 转换为 Call。先翻译 obj，再翻译 args。将他们都放入 use 集合中，然后创建 QuadCall。把左值的 temp 放入 def 集合中，然后打包为 QuadMoveCall。

2.5.2 Temp<-ExtCall

与上面类似，但是 obj 不放入 use，并且打包为 QuadExtCall 和 QuadMoveExtCall。

2.5.3 Temp<-Mem

将 temp 放入 def。把 src 转换为 Mem，翻译 Mem，得到 addr_term，放入 use。打包为 QuadLoad。

2.5.4 Temp<-Binop

src 转换为 Binop。先翻译 left 和 right。把 temp 放入 def。left_term 和 right_term 放入 use。如果 op 为 +，并且 type 为 PTR，则打包为指针计算 QuadPtrCalc。否则打包为 QuadMoveBinop。

2.5.5 其他

先翻译 src，如果 dst 为 mem，打包为 QuadStore，def 为空，use 为 src_term 和 dst_term。其余情况打包为 QuadMove。

2.6 Seq

逐个翻译 Stm。

2.7 LabelStm

打包为 QuadLabel。

2.8 Return

递归翻译 exp，得到 ret_term。放入 use 中，打包为 QuadReturn。

2.9 ExpStm

2.9.1 Call

先把 exp 转换为 Call。翻译 obj 得到 obj_term。翻译 args。将 obj_term 和 args 得到的 term 放入 use，打包为 QuadCall（不需要调用的返回值，所以不用 MoveCall）。

2.9.2 ExtCall

与上面类似，但是 use 中不含 obj_term。

2.9.3 其他

其余情况继续递归翻译，但是不对 output_term 进行处理（因为 ExpStm 本身就是“值被丢弃”）。

2.10 Binop

先分别翻译 left 和 right。调用 `temp_map->newtemp()` 新建一个中间值，根据 `binop->type` 创建一个 QuadTemp 作为 dst 并放入 def。left_term/right_term 放入 use。同样地，看是否为指针加法来判定是打包为 QuadPtrCalc 还是 QuadMoveBinop。然后把这个新建的中间值 dst 打包为 QuadTerm 返回。

2.11 Mem

先递归处理 mem 得到 addr_term 放入 use。同样新建一个 QuadTemp 中间值作为 dst 并放入 def。打包为 QuadLoad 语句放入 visit_result。然后返回 dst。

2.12 TempExp

打包为 QuadTerm 返回。

2.13 Eseq

先翻译和执行 Stm，保证副作用先运行。清空返回值，再翻译表达式 Exp 部分并得到返回值。

2.14 Name

先看看是否有 sname，有则为字符串标签，否则为数字标签，都打包为 QuadTerm 返回。

2.15 Const

打包为 QuadTerm，返回常量值。

2.16 Call

先翻译调用对象 obj（接收者）。然后翻译 args。将他们都放入 use。然后打包为 QuadCall，再新建一个中间量打包为 QuadTerm 作为 dst，加入到 def 中，最后打包为 QuadMoveCall。返回 dst。

2.17 ExtCall

与上面类似，但是 obj 不放入 use，打包为 QuadExtCall。最后打包为 QuadMoveExtCall。

3. git graph

- Merge branch 'master' into d...  
- hw5 finished Starw1sh
- HW6 README minor edit 王晓阳
- HW6 updated 王晓阳
- HW6 updated 王晓阳
- HW6 test files 王晓阳
- HW6 remove build folder 王晓阳
- HW6 added 王晓阳
- HW6 added 王晓阳
- runquad-ubuntu-aarch64 updated 王晓阳
- runquad Makefile updated xseanwang
- HW5 vendor/runquad added amd64 versi...
- hw4 handin Starw1sh
- Merge branch 'master' into dev Starw1sh
- Updated runquad-macos-silicon 王晓阳
- HW5 vendor/runquad Makefile change...
- HW5 vendor/runquad updated 王晓阳
- HW5 test delete llvm file xywang2772
- HW5: tests added xml.quad output files x...
- HW5 tests added 王晓阳
- add HW1 hidden tests leo
- HW5 Added 王晓阳